AWS CloudFormation and its Stack Creation Demo

9 min read · Jul 10, 2023

Satya Mounika Mushini Follow

Listen Share More

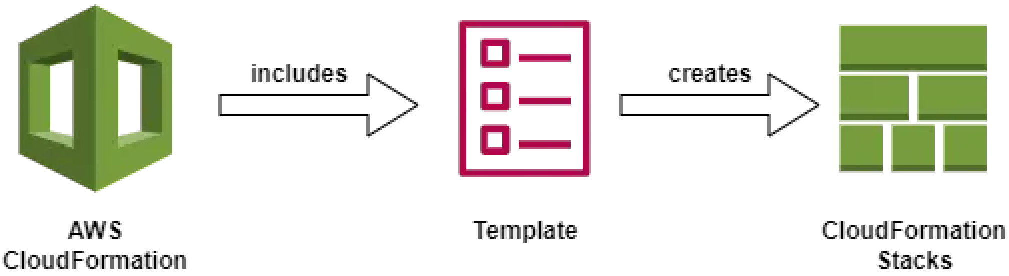
In this article, I will explain about one of the Amazon Web Services (AWS) Management Tools, i.e. **CloudFormation.**

AWS CloudFormation:

AWS CloudFormation was first introduced by Amazon Web Services (AWS) in 2011. Its a service that allows one to define and manage the AWS infrastructure resources in a declarative way.

CloudFormation uses a template-based approach, where we define the Infrastructure as a Code(IaaC) using JSON or YAML template file. In this template file, we will define the infrastructure, i.e. resources and configurations. CloudFormation takes care of creating, updating and deleting of resources.

In AWS CloudFormation, a Stack can be defined by a CloudFormation template, which is a JSON or YAML file that describes the desired state of the infrastructure that we want to create. It basically consists the resources, the configurations and dependencies for the resources.



A CloudFormation Sample Template:

```
---
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Sample CloudFormation Template'

Resources:
  EC2Instance:
    Type: 'AWS::EC2::Instance'
    Properties:
      ImageId: ami-********
      InstanceType: t2.micro
      KeyName: keypair-name
      SecurityGroupIds:
        - sg-*****
      SubnetId: subnet-*****
  VPC:
    Type: 'AWS::EC2::VPC'
  ----
  InternetGateway:
    Type: 'AWS::EC2::InternetGateway'
  ----
  Subnet:
    Type: 'AWS::EC2::Subnet'
  ----
  Route:
    Type: 'AWS::EC2::Route'
  ----
  S3Bucket:
    Type: 'AWS::S3::Bucket'
  ----
```

- **AWSTemplateFormatVersion:** This specifies the version of the CloudFormation template.
- **Description:** Provides the purpose of the template.

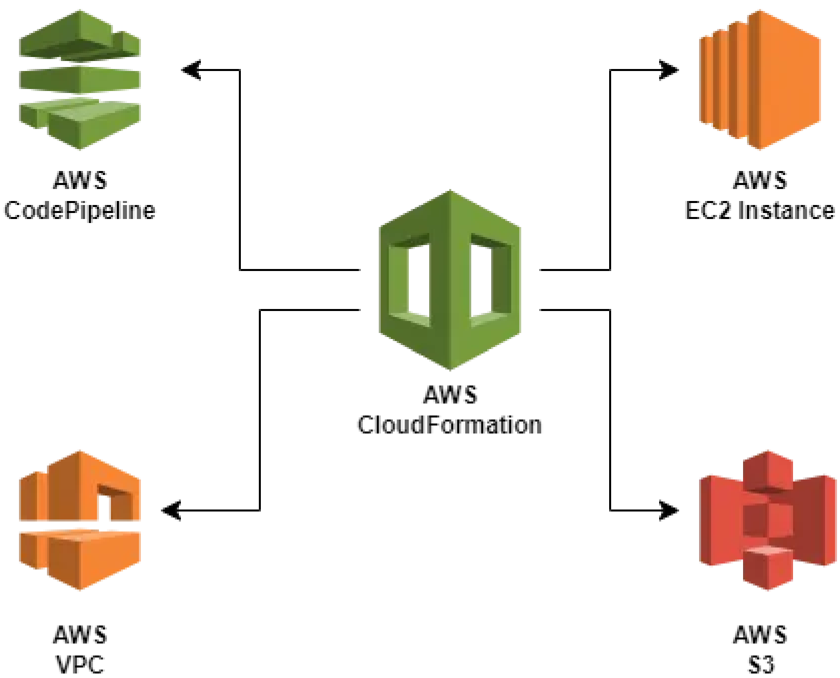
- **Resources:** We will define the resources that we want to create.
- **Type:** This specifies the resource type being created.
- **Properties:** We specify the properties and configurations of the resource. Like for example, for an EC2 Instance, we add ImageID, Instance Type, keyname, Security Groups, Port numbers, Availability zones, CIDR Blocks, Tags etc.

We have n-no. of ways to deploy CloudFormation stacks, that are namely:

1. AWS Management Console
2. AWS Command Line Interface (CLI)
3. AWS CloudFormation API
4. AWS SDKs
5. AWS CloudFormation Templates
6. AWS CloudFormation StackSets

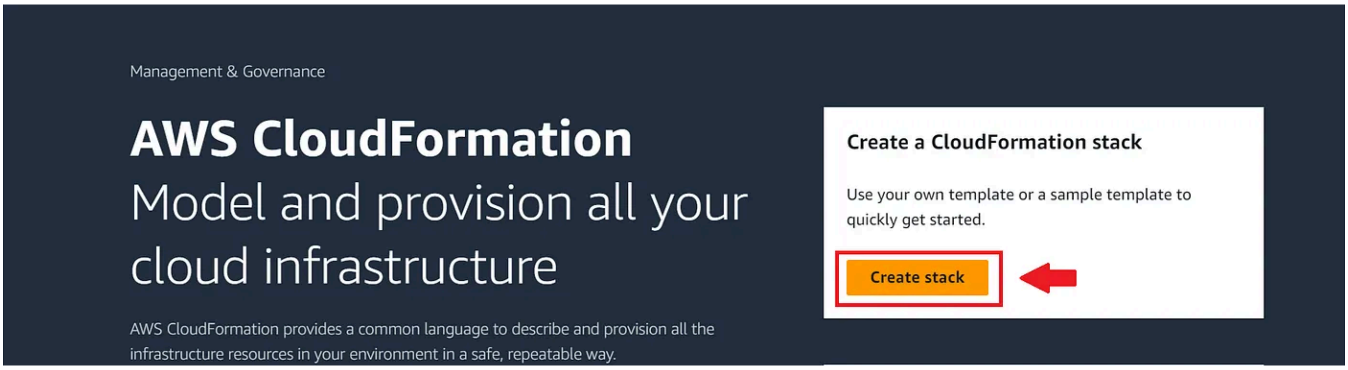
In this article, we will learn how to create Stacks through AWS Management Console through Template File and S3 Bucket and AWS Command Line Interface (CLI).

AWS CloudFormation Stack Creation using Template File:

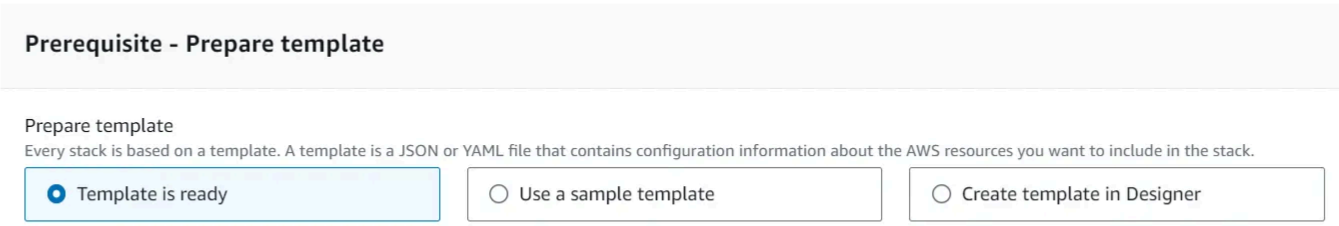


Resources that are being created using CloudFormation

1. Login to your AWS account and Navigate to **CloudFormation.** Click on **Create Stack.**



2. For Prerequisite-Prepare Template, select **Template Ready**.



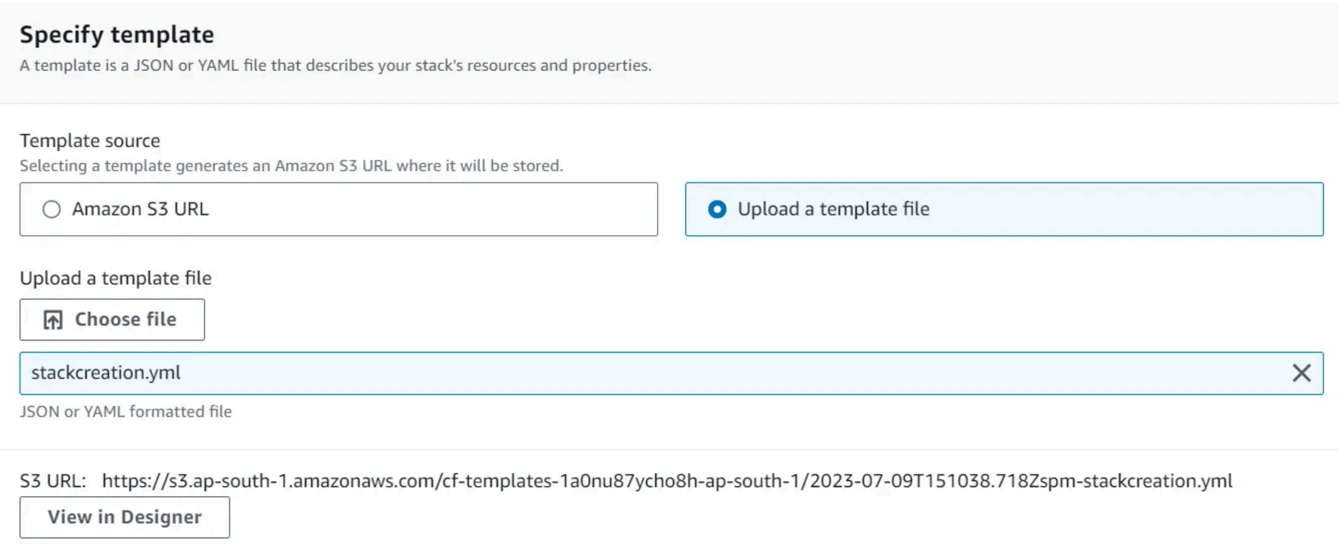
3. In Specify Template, select **Upload a Template File** and upload the YAML or JSON template file that consists of the script for the creation of resources. Select **Choose File** and upload the template.

```
AWSTemplateFormatVersion: "2010-09-09"
Resources:
  MyVPC:
    Type: "AWS::EC2::VPC"
    Properties:
      CidrBlock: "10.0.0.0/16"
      Tags:
        - Key: Name
          Value: "Stack-VPC"

  MyInternetGateway:
    Type: "AWS::EC2::InternetGateway"

  MyVPCGatewayAttachment:
    Type: "AWS::EC2::VPCGatewayAttachment"
    Properties:
      VpcId: !Ref MyVPC
      InternetGatewayId: !Ref MyInternetGateway

  MySubnet:
```

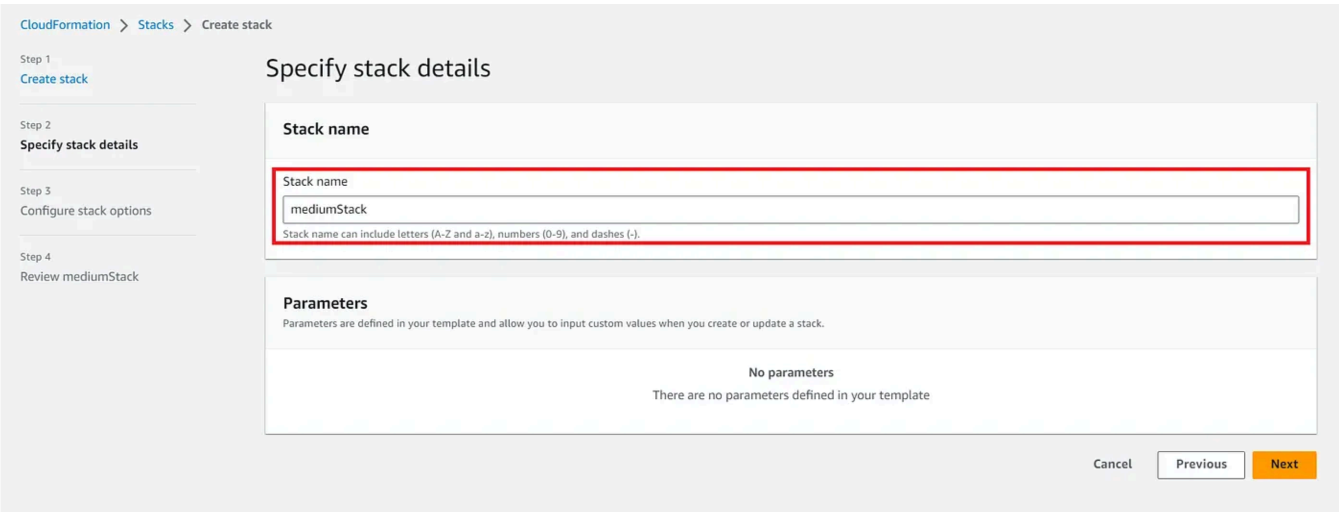
```
    Type: "AWS::EC2::Subnet"
    Properties:
      VpcId: !Ref MyVPC
      CidrBlock: "10.0.0.0/24"
      AvailabilityZone: "ap-south-1a"
      Tags:
        - Key: Name
          Value: "MySubnet"

  MyRouteTable:
    Type: "AWS::EC2::RouteTable"
    Properties:
      VpcId: !Ref MyVPC
      Tags:
        - Key: Name
          Value: "MyRouteTable"

  MyRoute:
    Type: "AWS::EC2::Route"
    DependsOn: MyVPCGatewayAttachment
    Properties:
      RouteTableId: !Ref MyRouteTable
      DestinationCidrBlock: "0.0.0.0/0"
      GatewayId: !Ref MyInternetGateway

  MySubnetRouteTableAssociation:
    Type: "AWS::EC2::SubnetRouteTableAssociation"
    Properties:
      SubnetId: !Ref MySubnet
      RouteTableId: !Ref MyRouteTable

  MySecurityGroup:
    Type: "AWS::EC2::SecurityGroup"
    Properties:
      GroupDescription: "My security group"
      VpcId: !Ref MyVPC

  MyEC2Instance:
    Type: "AWS::EC2::Instance"
    Properties:
      InstanceType: "t2.micro"
      SecurityGroupIds:
        - !Ref MySecurityGroup
      SubnetId: !Ref MySubnet
      ImageId: "ami-0607784b46cbe5816"
      KeyName: "awskey"
      Tags:
        - Key: Name
          Value: "Stack-EC2"

  MyS3Bucket:
    Type: "AWS::S3::Bucket"
```

```
      Properties:
        BucketName: "mounikabucket2612"
```

4. After you upload the file, an S3 URL will be generated for the same. This will be the Template URL for the stack creation. After this, click **Next**.



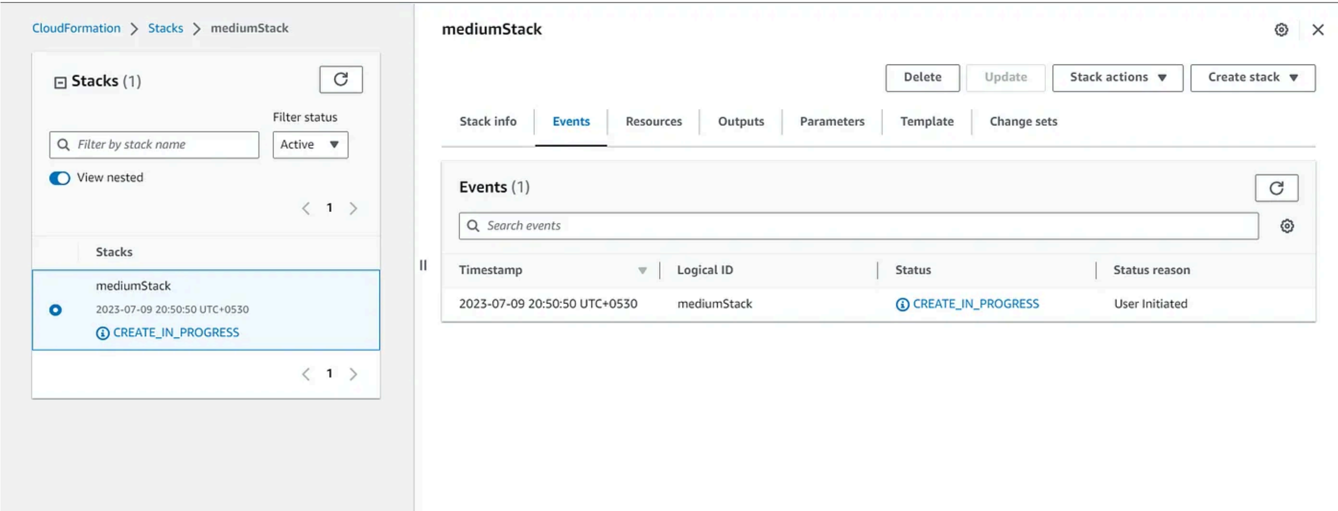
5. Provide a name for the Stack. Click Next.



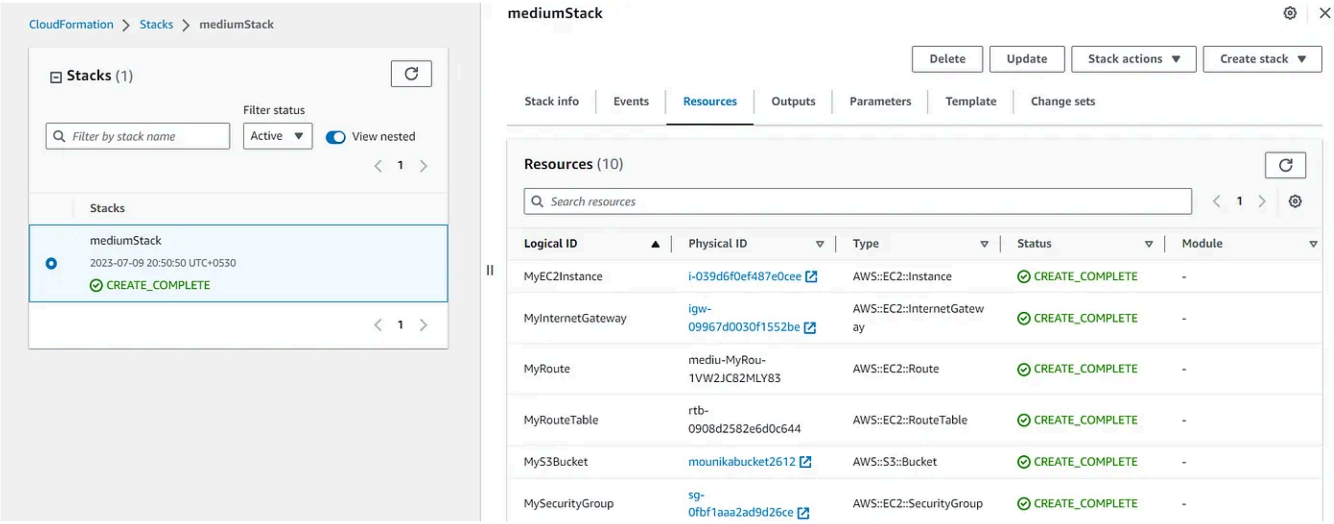
6. In **Configure Stack Options**, consider all the default values and click Next.

7. You will be given a preview of all the details related to the Stack creation. In this except the Template with a URL, other fields remain empty or possess a default value. Click **Submit**.

Resource Creation is In Progress:



Resource Creation Completed:

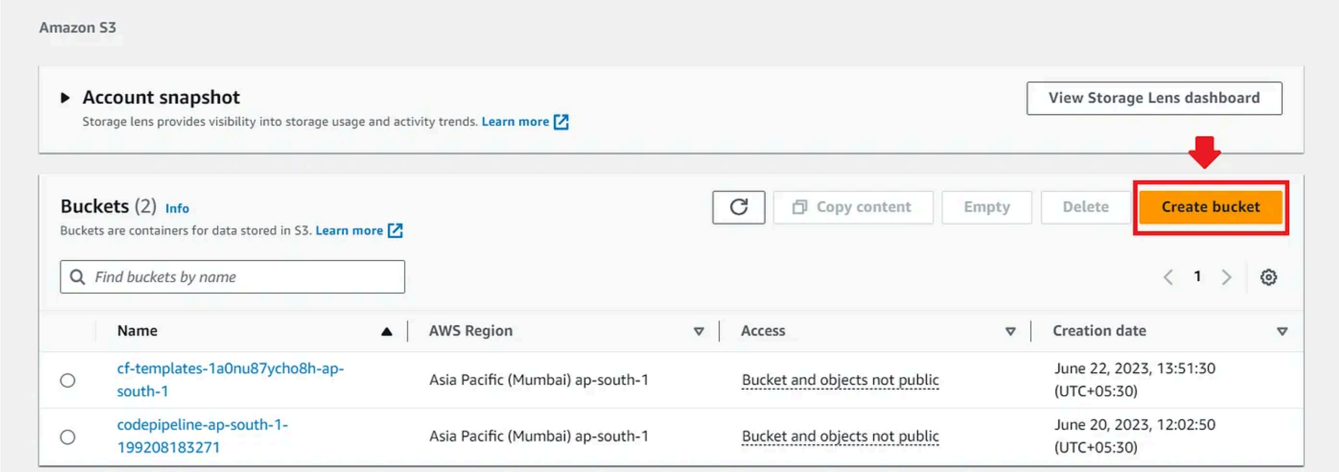


Congratulations! Stack has been created successfully.

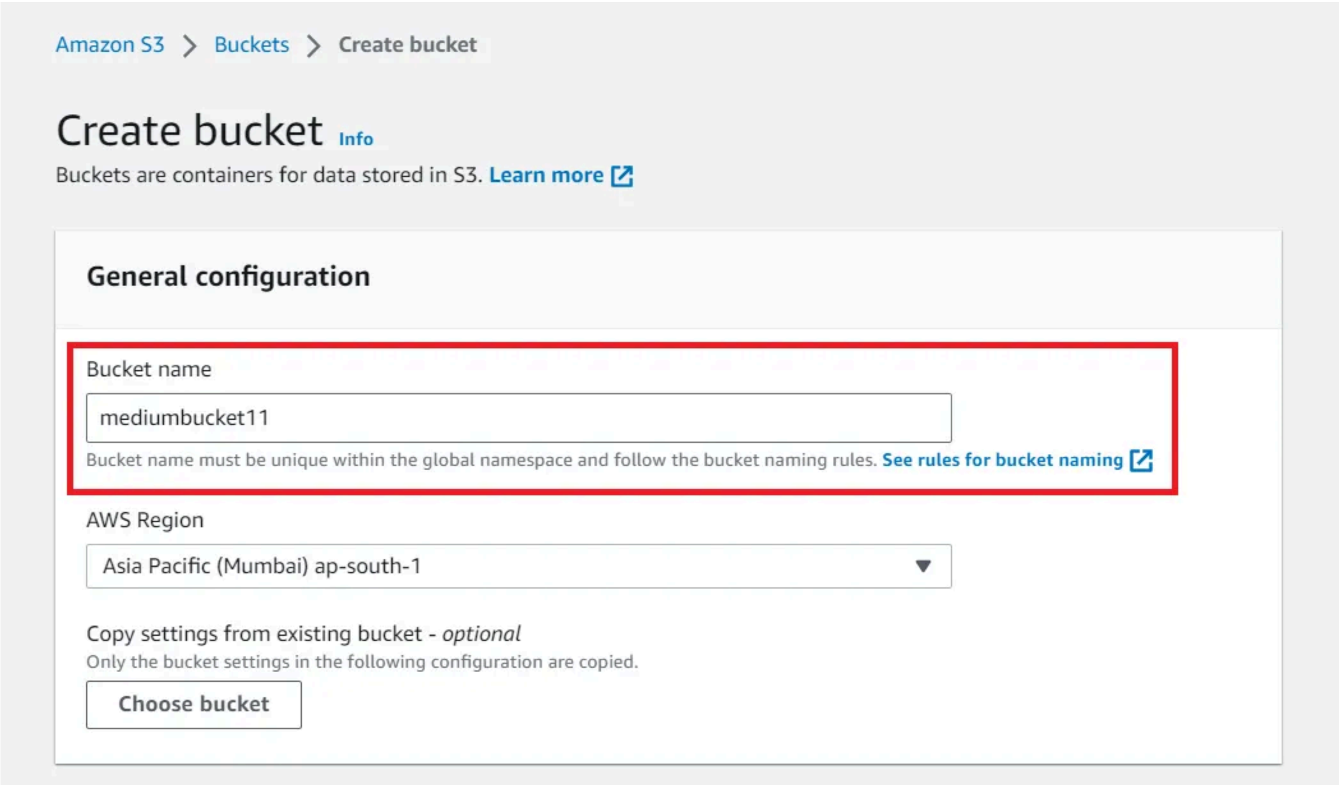
Resources Created with the Template : EC2 Instance, S3, VPC, Subnet, RouteTable, Internet Gateway etc.

AWS CloudFormation Stack Creation using S3 Bucket:

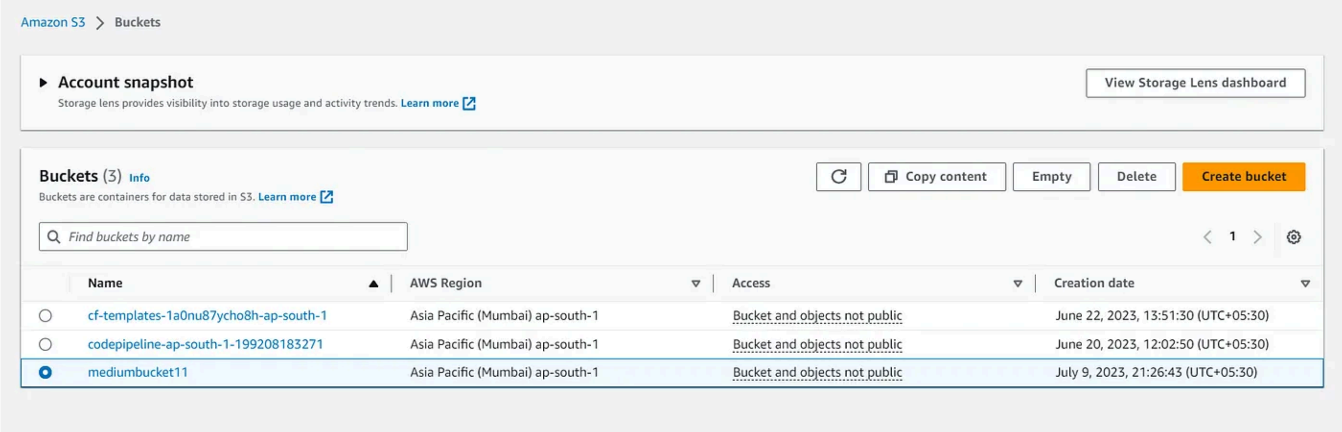
1. Login to your AWS account and Navigate to S3 page. Click on **Create Bucket.**



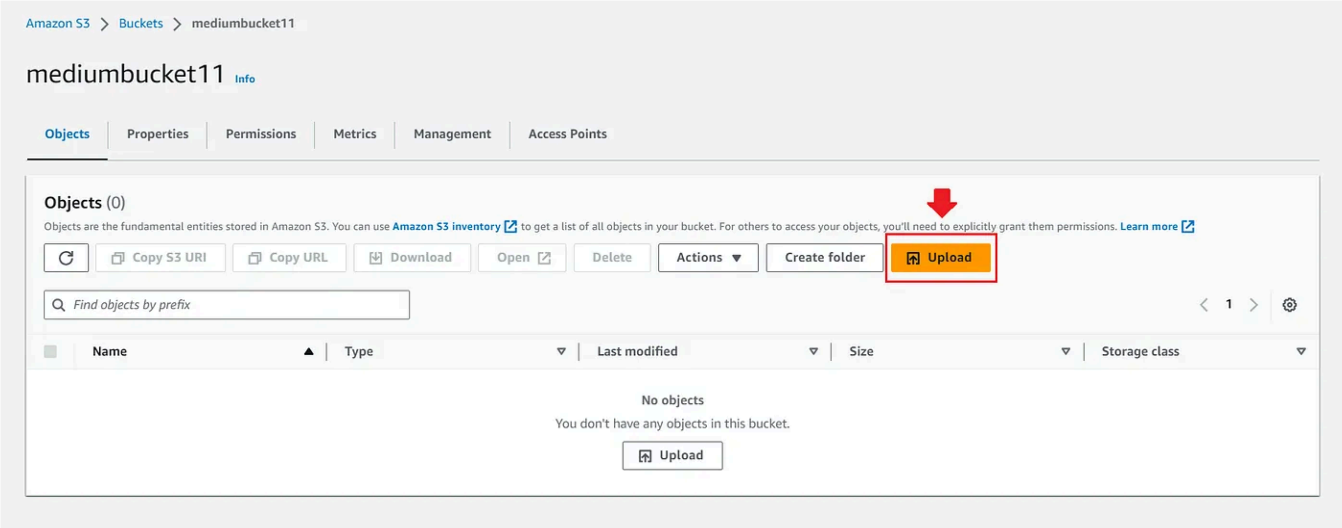
change it.



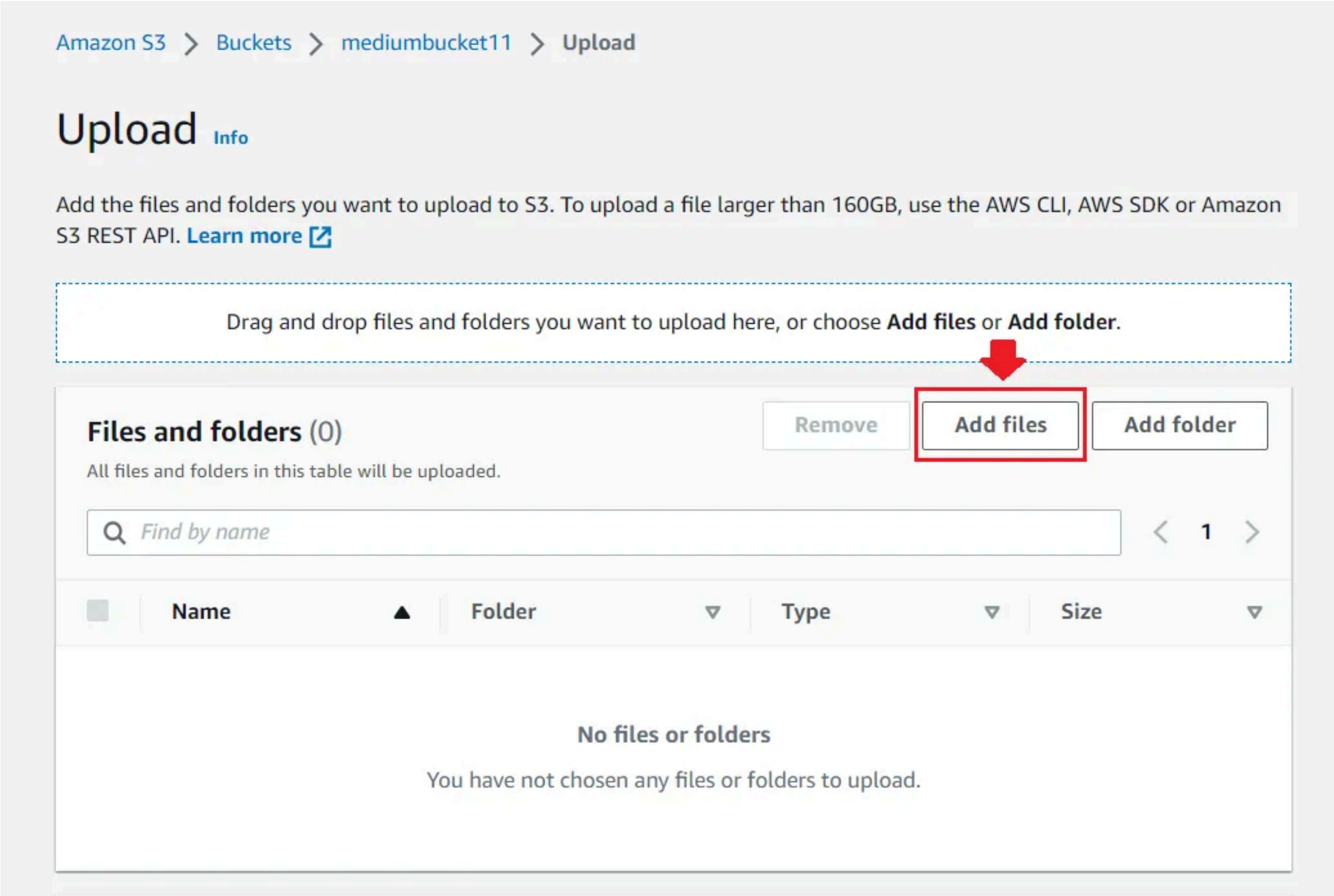
3. A Bucket has been created. In this bucket, we can upload the Template files to create a Stack.



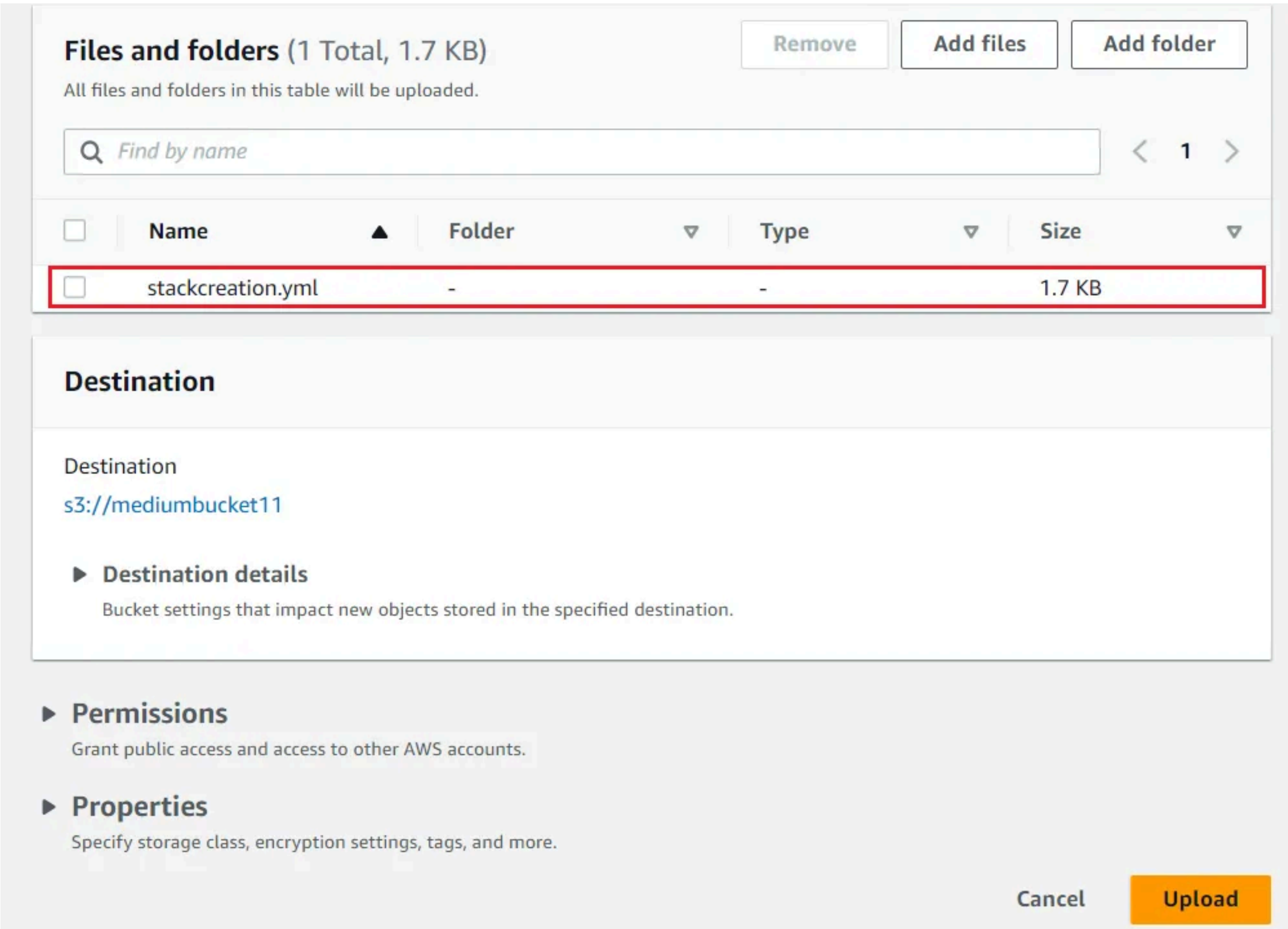
4. Click on the created Bucket and select **Upload**.



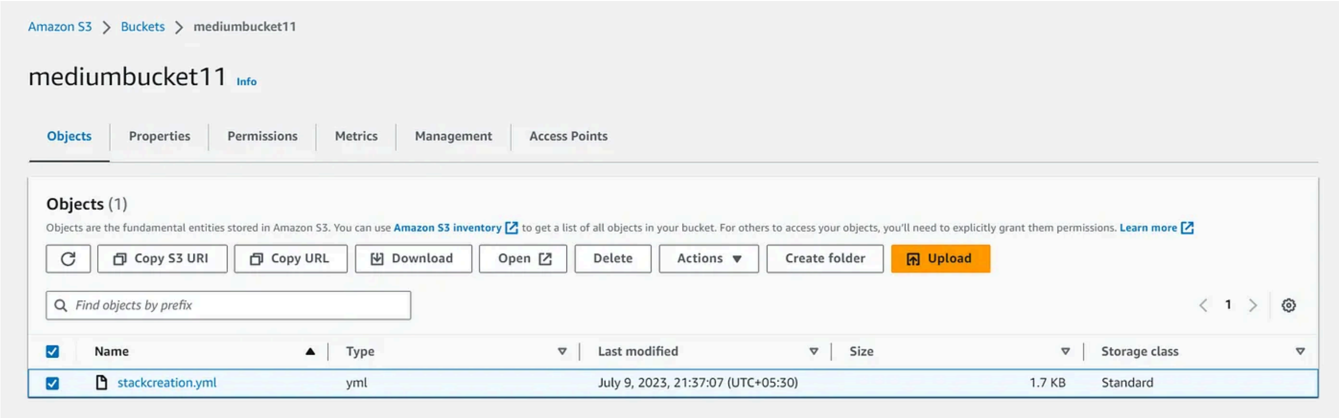
5. Select **Add files** and add the Template file through it.



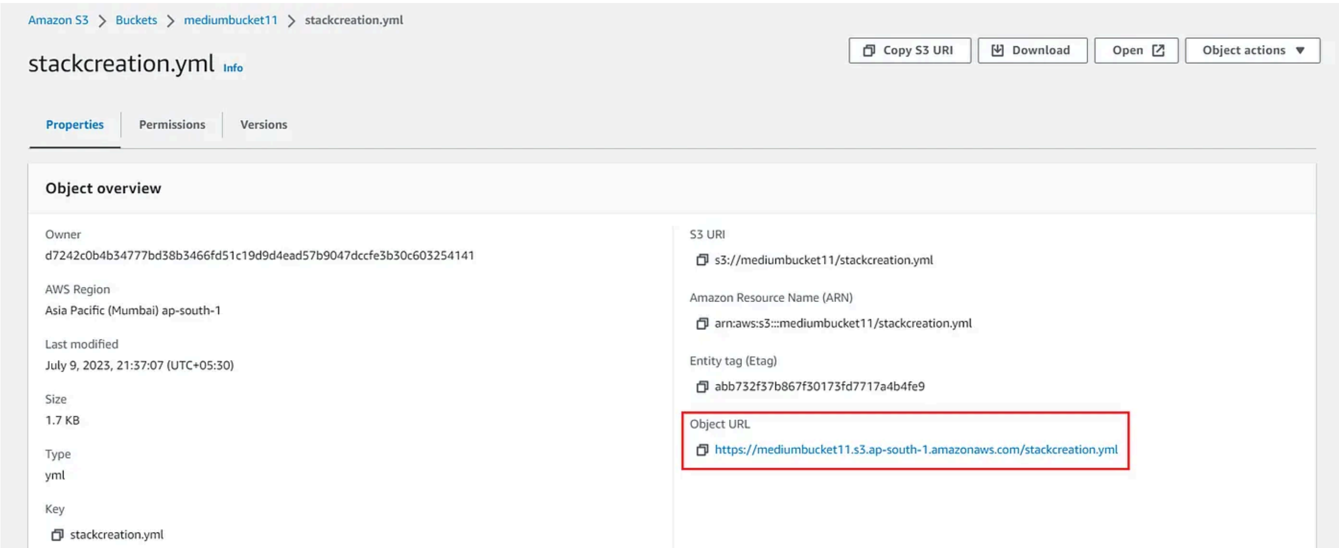
6. After adding the file, Click on **Upload**.



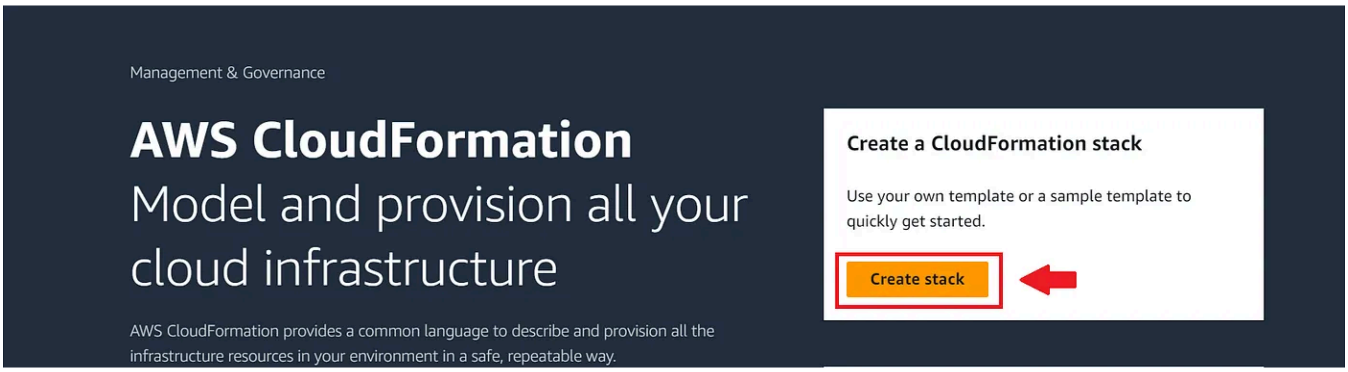
7. File has been uploaded successfully in the bucket.



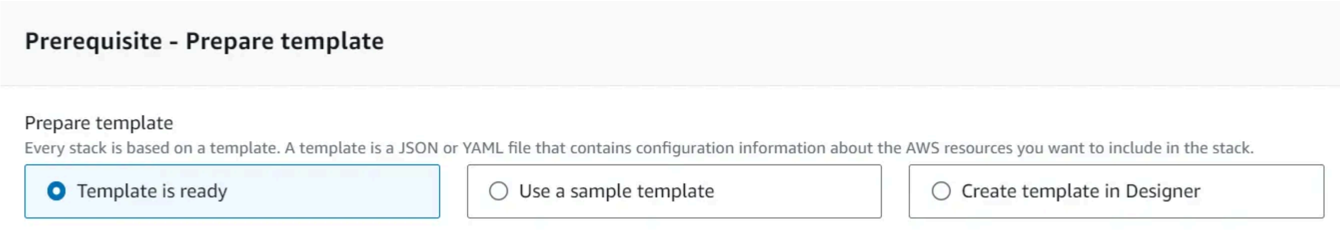
8. Click on the file that has been uploaded. You will be directed to a page that consists of the properties details of the uploaded file. In that the Object URL will be specified. This will be our **Amazon S3 Template URL.**



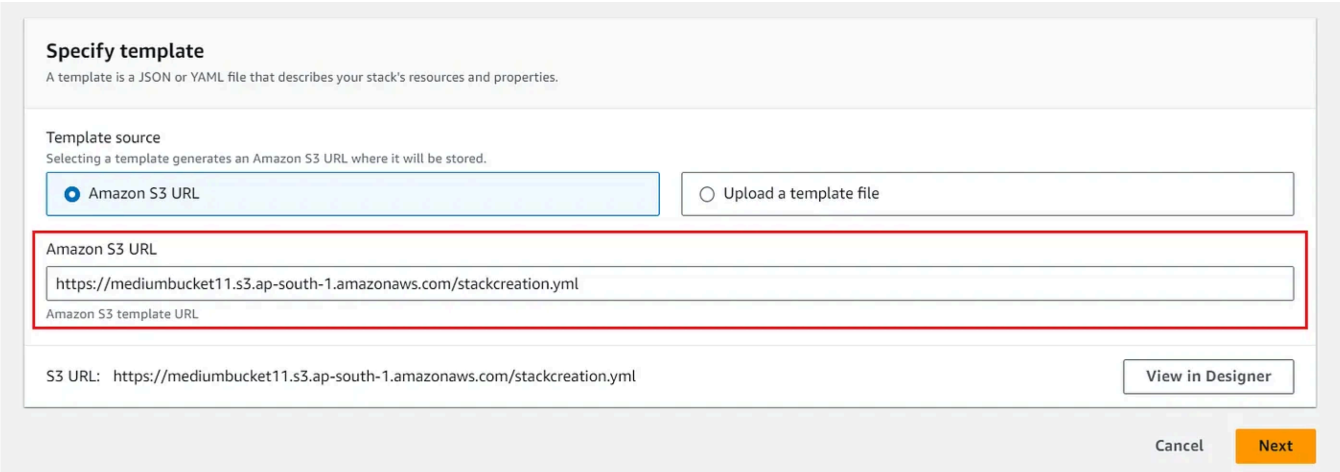
9. Now got to **CloudFormation** page and Create click on **Create Stack**.



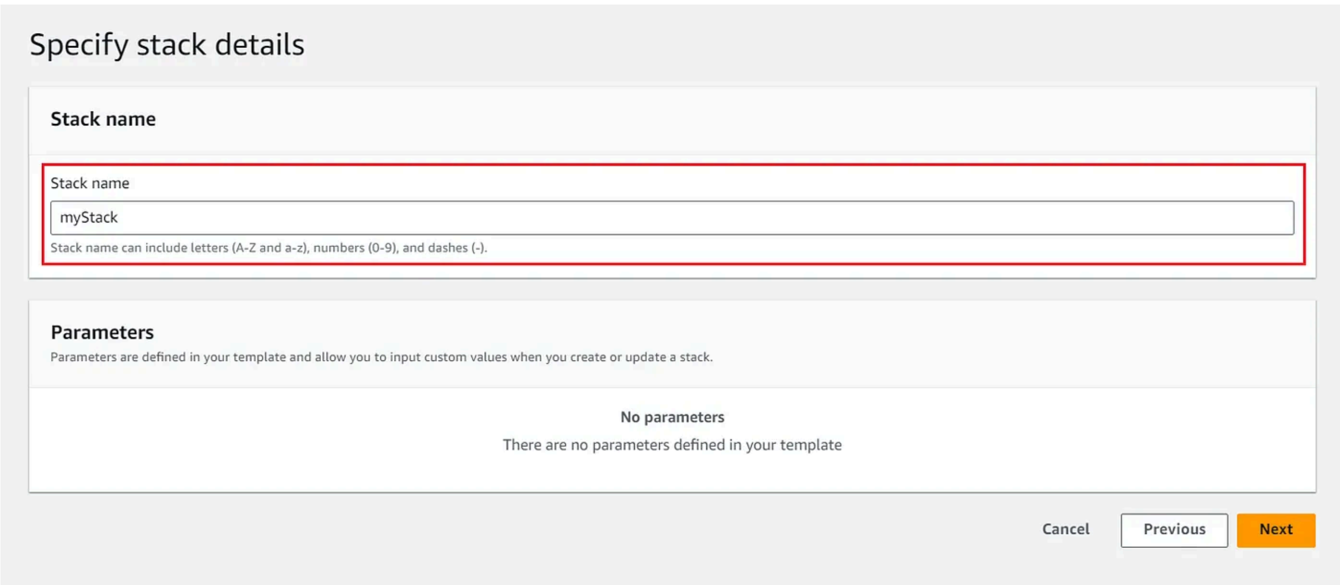
10. For Prerequisite-Prepare Template, select **Template Ready**.



11. Select Amazon S3 URL and in the field, provide the Object URL of the template that you have uploaded in the S3 Bucket (Check Point 8). Click on **Next.**



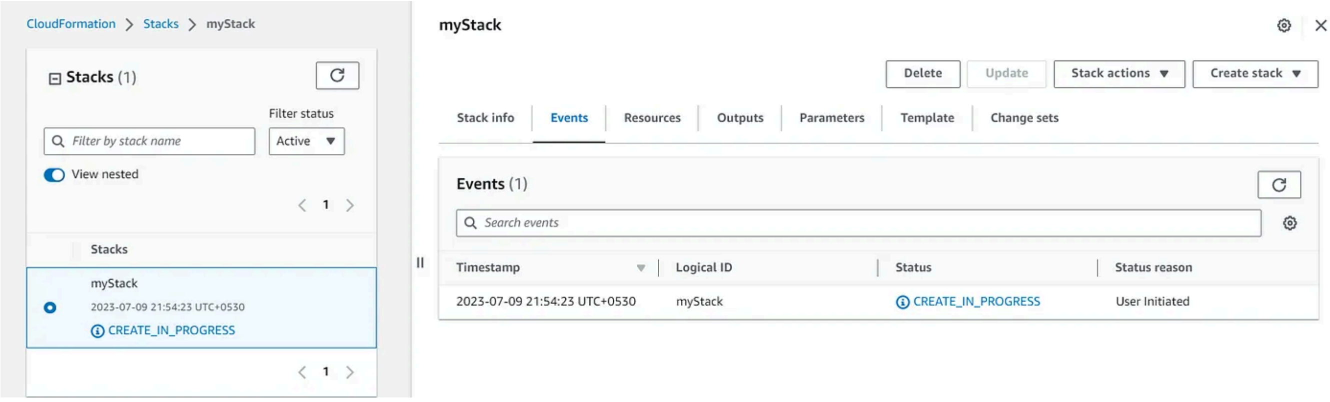
12. Provide a Stack name. Click on **Next.**



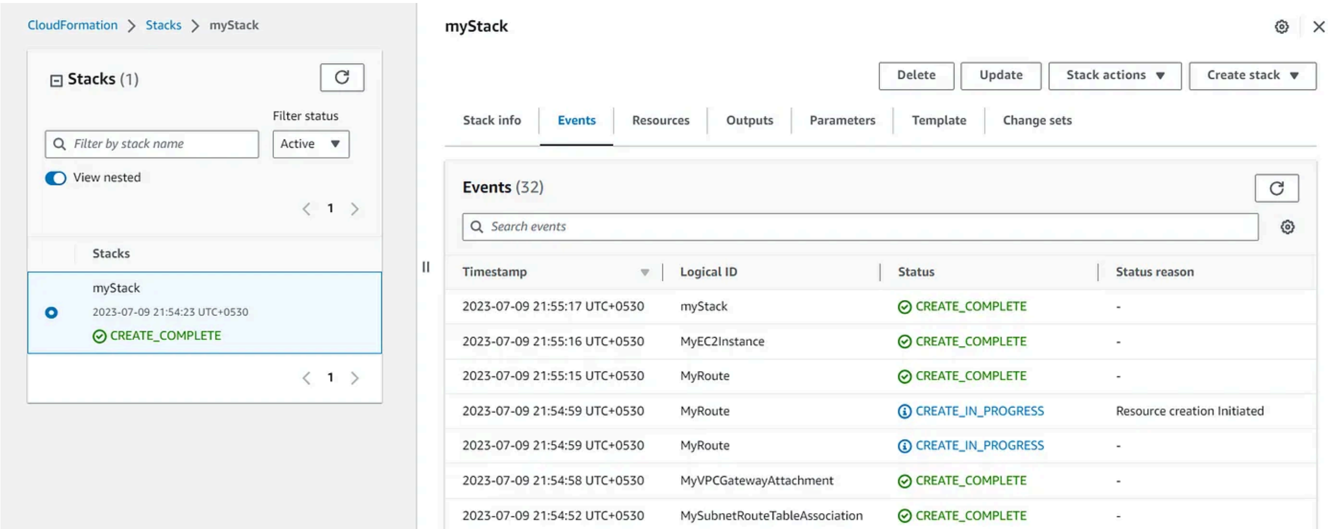
13. In **Configure Stack Options**, consider all the default values and click Next.

14. You will be given a preview of all the details related to the Stack creation. In this except the Template with a URL, other fields remain empty or possess a default value. Click **Submit.**

Resource Creation is In Progress:



Resource Creation Completed:



Congratulations! Stack has been created successfully.

Resources Created with the Template : EC2 Instance, S3, VPC, Subnet, RouteTable, Internet Gateway etc.

AWS CloudFormation — Stack Creation using AWS CLI:

1. First step is to download the AWS CLI. For Windows (64-bit), run the following command:

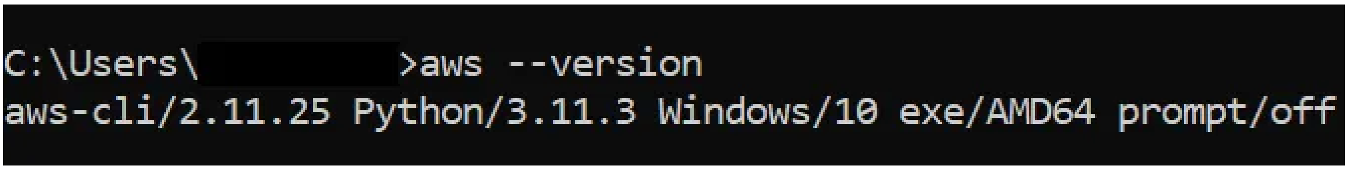
```
C:\> msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

For other Operating Systems, You can check this page : [Amazon CLI]

2. To confirm the installation, open CMD from the Start Menu and run the following command:

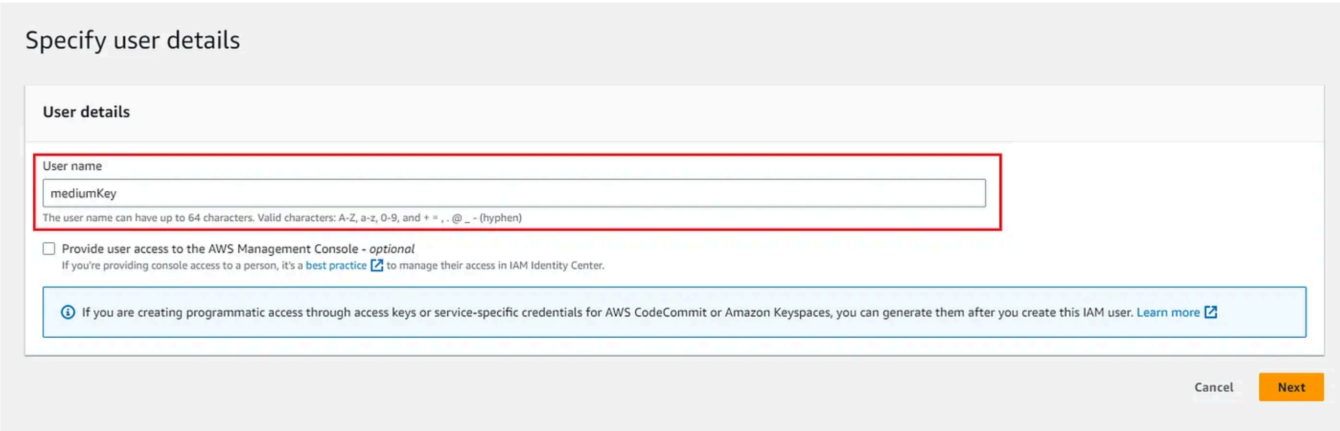
```
C:\> aws --version
```

Output:

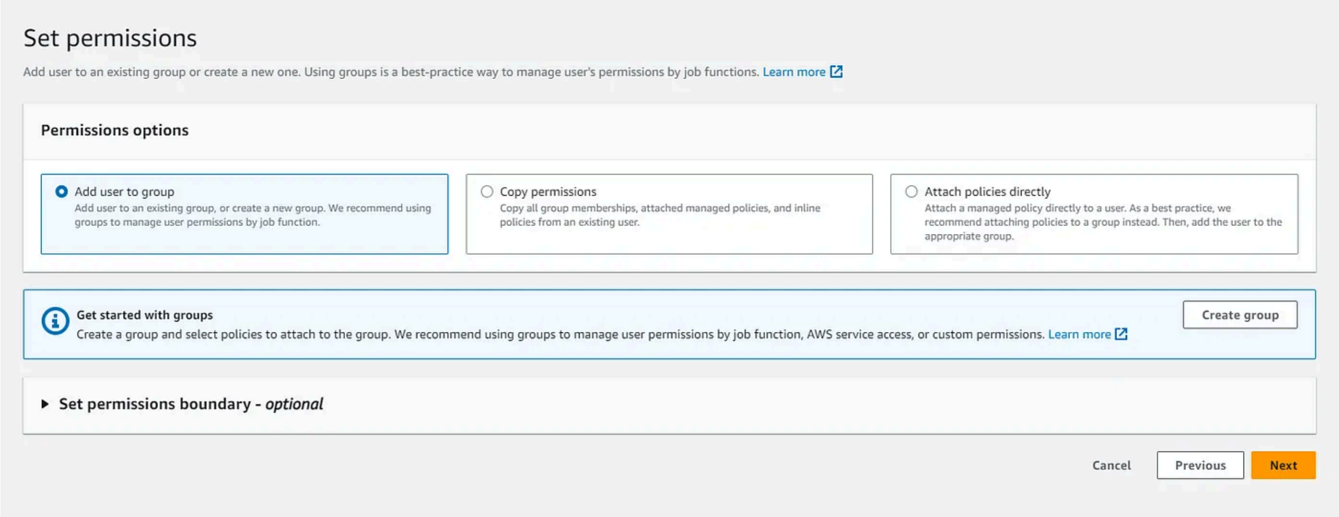


3. Next, Create an IAM role. Go to **IAM** in AWS Console Management and got to **Users.** Now click on **Add Users.**

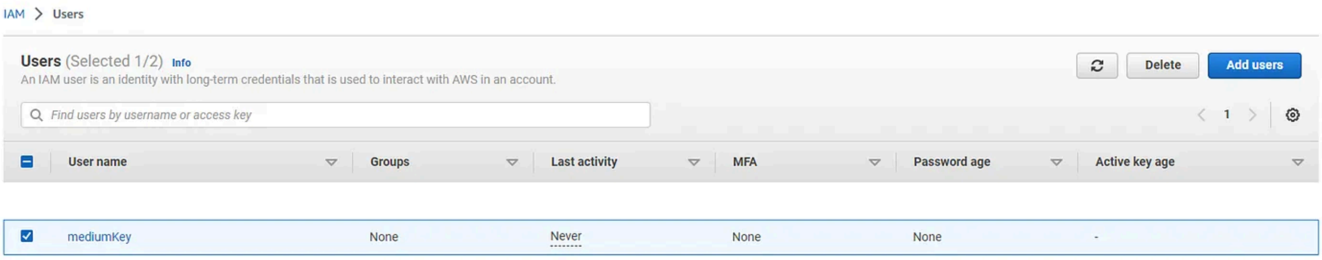
4. Provide a User Name. Click on **Next.**



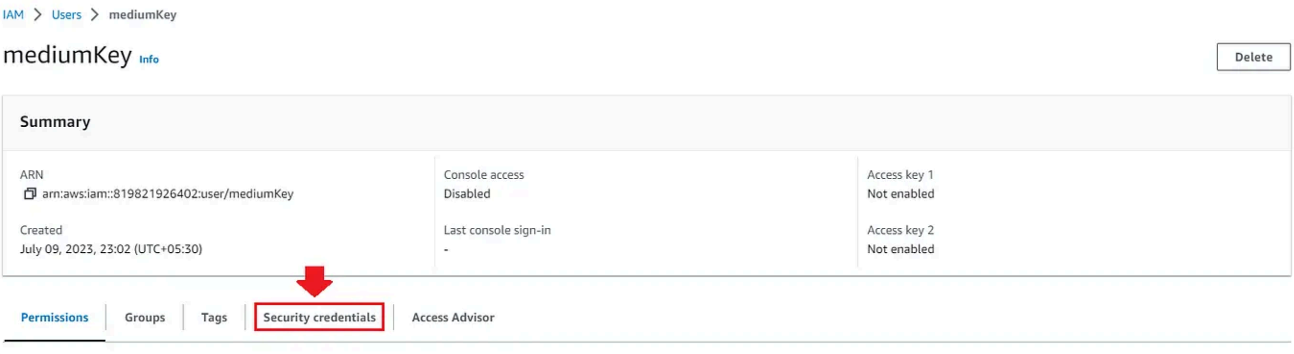
5. In Set Permissions, Select **Add User to Group.** Click **Next.**



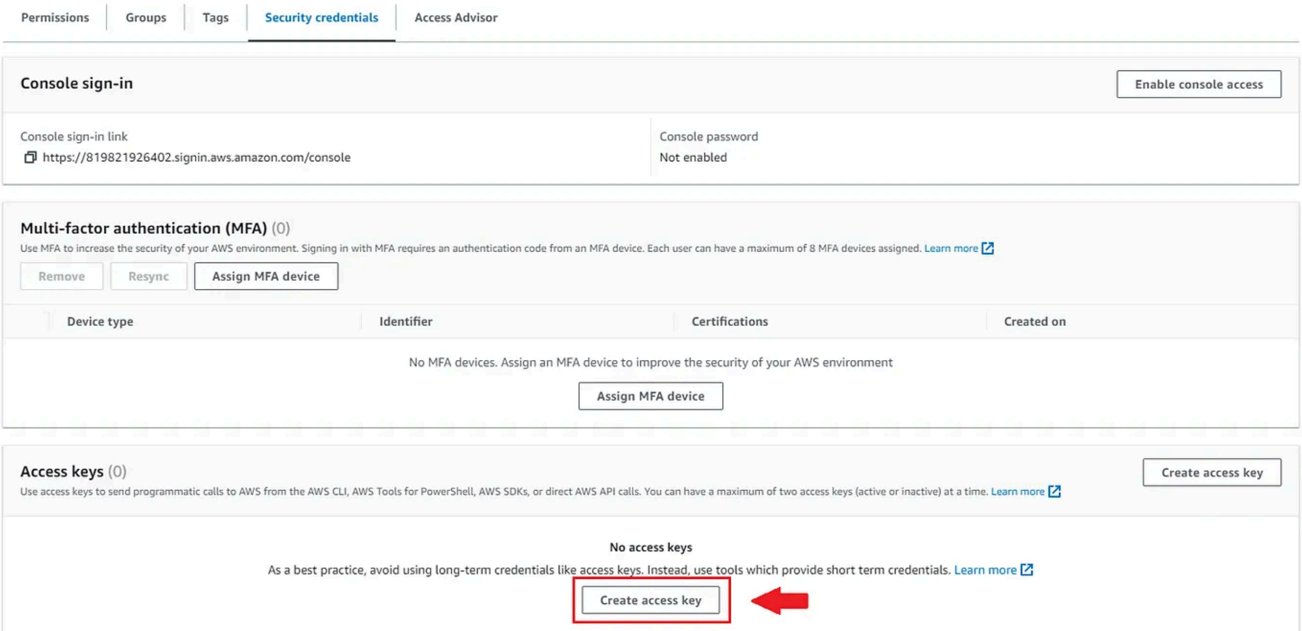
6. Review all the details and click on **Create User**. A new User has been created and added to the Users List.



7. Click on the created User and Go to **Security Credentials**.



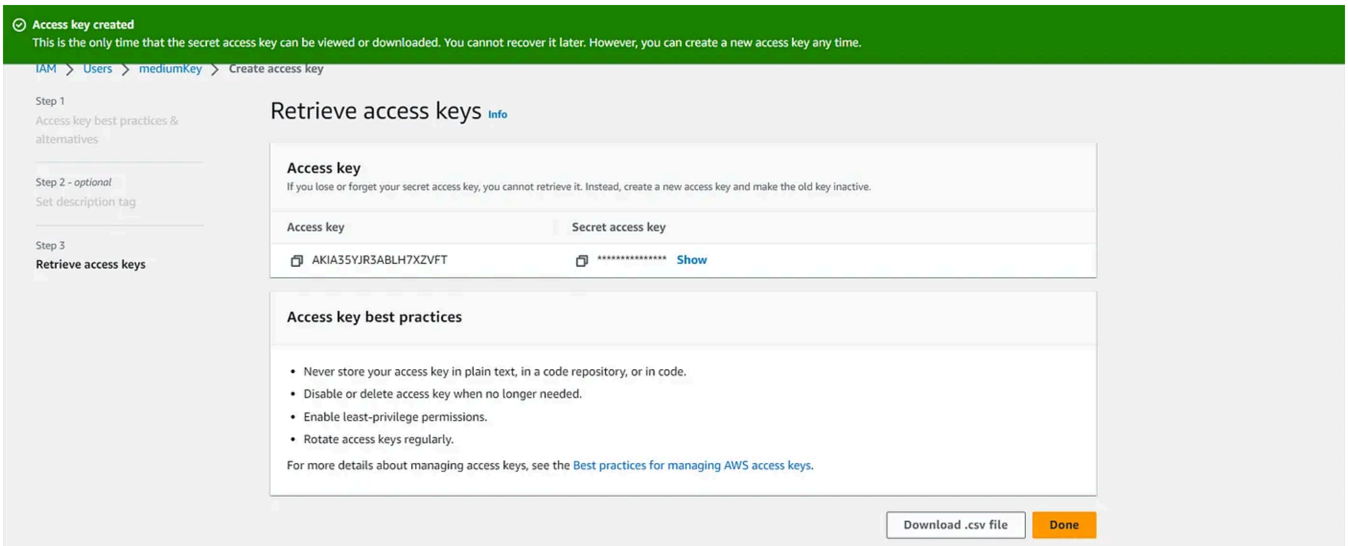
8. Click on Create Access Key.



9. Select the Use case : **Command Line Interface (CLI)** and tick the Confirmation checkbox and click **Next.**

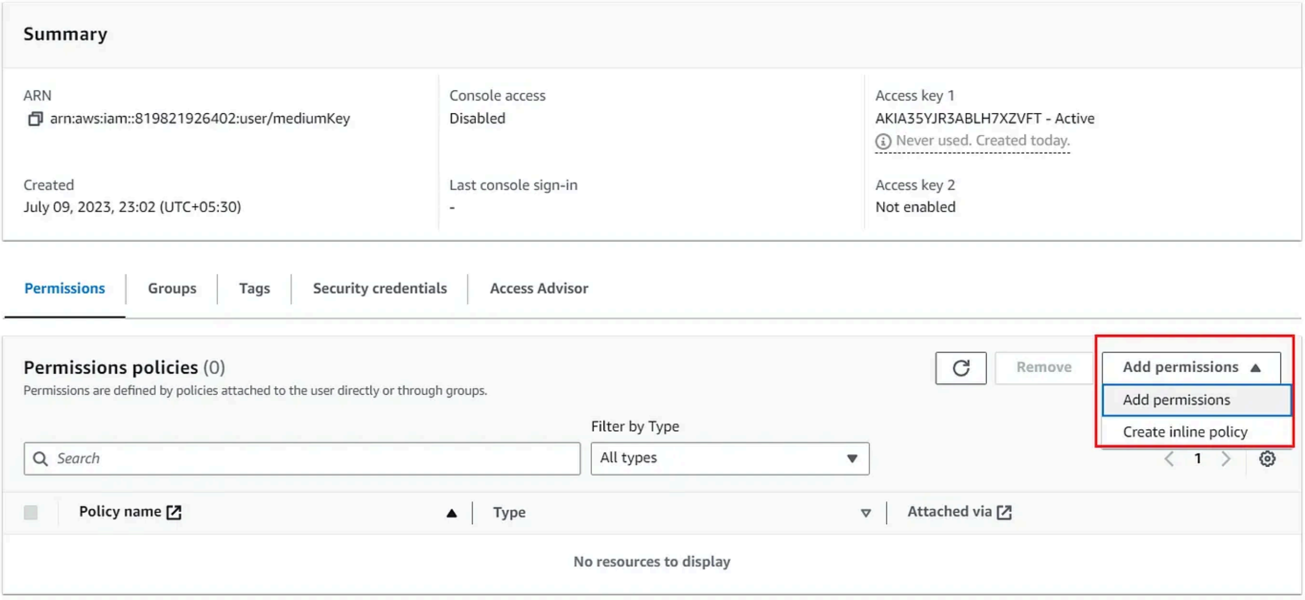
Skip the description tag and Click **Create Access Key**.

Output:

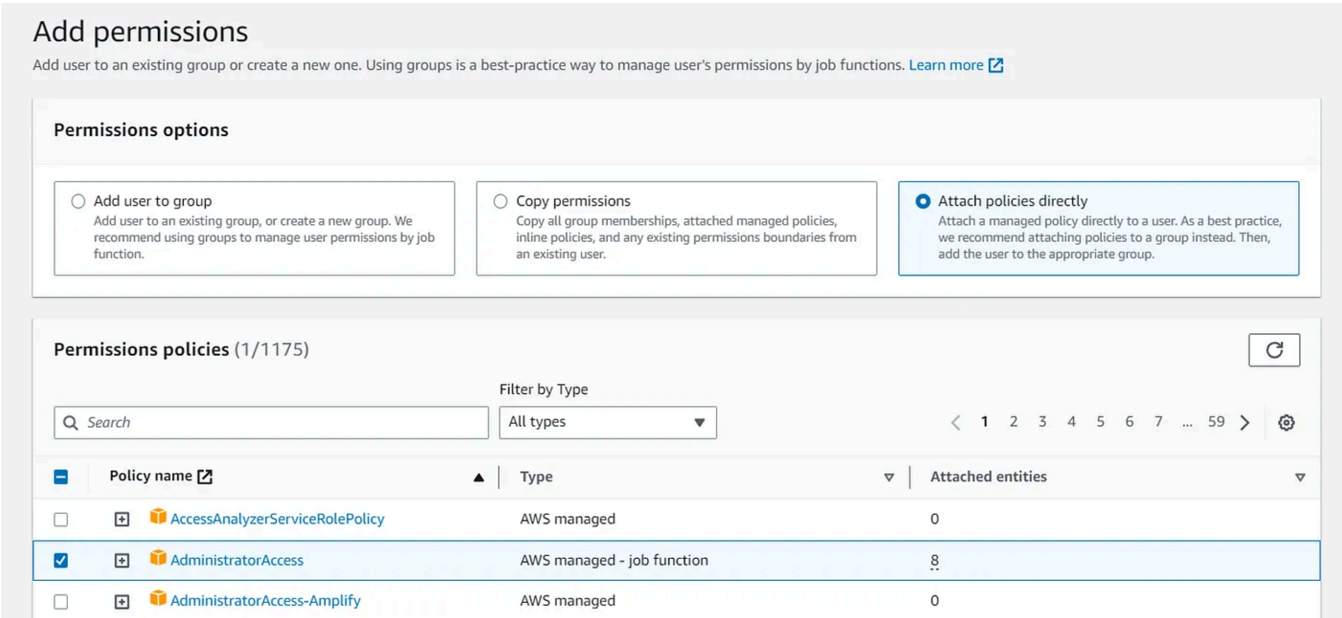


Try to store the Access key and Secret Access Key on a notepad as you may not be able to recover it later. You can also do this by clicking on **download.csv** file.

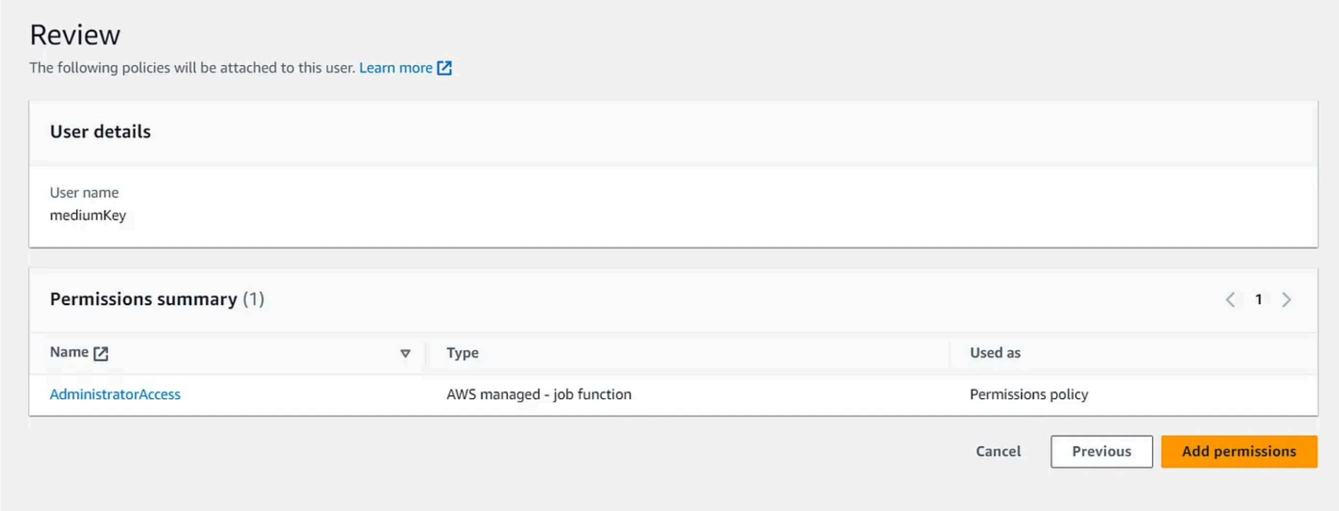
10. Got to Permissions of the User that you have created in IAM and click **Add Permissions : Add permission**



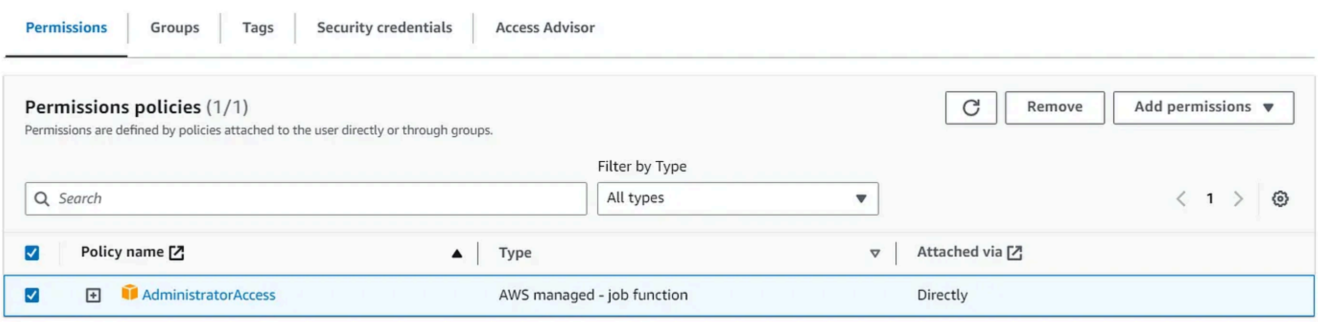
11. In Add Permissions, Select **Attach Policies directly** and choose **AdministratorAccess.** Click **Next.** (AdministratorAccess permission is given just for demo purpose. Usually, we provided permissions related to the resources we created.



12. Review all the details and click **Add Permissions.**



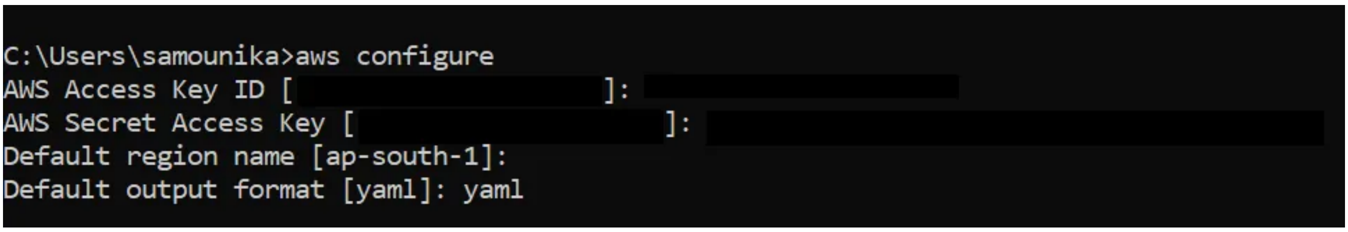
It will look like this:



13. Now, Go to CMD and run the following command:

```
aws configure
```

14. You need to provide the Access key, Secret Access Key that you have created here. Please specify the region and the kind of output format i.e. JSON or YAML.



Run the following command to create the stack through CLI. (run the command where the YAML file is present)

AWS CLI Commands :

1. **Create Stack:** Creates a stack as specified in the template

```
aws cloudformation create-stack --stack-name myStackCreation --template-body fi
```

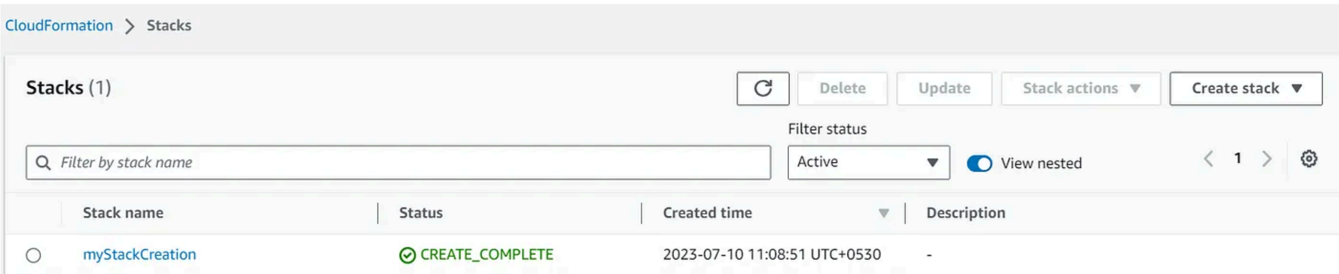
myStackCreation : Its the stack-name.

file://stackcreation.yml : Provide the name of the file. As my file is saved on the desktop, I have provided just the file name.

Output:

```
C:\Users\samounika\Desktop>aws cloudformation create-stack --stack-name myStackCreation --template-body file://stackcreation.yml
StackId: arn:aws:cloudformation:ap-south-1:819821926402:stack/myStackCreation/edf63520-1edf-11ee-9406-024c8cd7537c
```

Stack Creation



Creation Complete

2. **Describe Stacks:** It will return the description of the stack

```
aws cloudformation describe-stacks --stack-name myStackCreation
```

```
C:\Users\samounika\Desktop>aws cloudformation describe-stacks --stack-name myStackCreation
Stacks:
- CreationTime: '2023-07-10T05:09:21.520000+00:00'
  DisableRollback: false
  DriftInformation:
    StackDriftStatus: NOT_CHECKED
  EnableTerminationProtection: false
  NotificationARNs: []
  RollbackConfiguration: {}
  StackId: arn:aws:cloudformation:ap-south-1:819821926402:stack/myStackCreation/edf63520-1edf-11ee-9406-024c8cd7537c
  StackName: myStackCreation
  StackStatus: CREATE_COMPLETE
  Tags: []
```

3. **Template Summary:** Returns the summary of the template

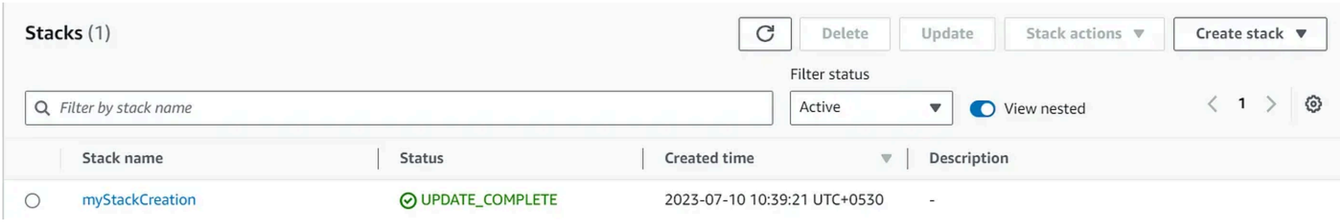
```
aws cloudformation get-template-summary --template-body file://stackcreation.ym
```

```
Parameters: []
ResourceIdentifierSummaries:
- LogicalResourceIds:
  - MyVPC
  ResourceIdentifiers:
  - VpcId
  ResourceType: AWS::EC2::VPC
- LogicalResourceIds:
  - MyRouteTable
  ResourceIdentifiers:
  - RouteTableId
  ResourceType: AWS::EC2::RouteTable
- LogicalResourceIds:
  - MySubnetRouteTableAssociation
  ResourceIdentifiers:
  - Id
  ResourceType: AWS::EC2::SubnetRouteTableAssociation
- LogicalResourceIds:
  - MyS3Bucket
  ResourceIdentifiers:
  - BucketName
  ResourceType: AWS::S3::Bucket
- LogicalResourceIds:
  - MyEC2Instance
  ResourceIdentifiers:
  - InstanceId
  ResourceType: AWS::EC2::Instance
- LogicalResourceIds:
  - MyInternetGateway
  ResourceIdentifiers:
  - InternetGatewayId
-- More  --
```

4. **Update Stack:** Updates an existing stack as specified in template. (I have changed the bucket name, to show a demo)

```
aws cloudformation update-stack --stack-name myStackCreation --template-body fi
```

```
C:\Users\samounika\Desktop>aws cloudformation update-stack --stack-name myStackCreation --template-body file://stackcreation.yml
StackId: arn:aws:cloudformation:ap-south-1:819821926402:stack/myStackCreation/edf63520-1edf-11ee-9406-024c8cd7537c
```

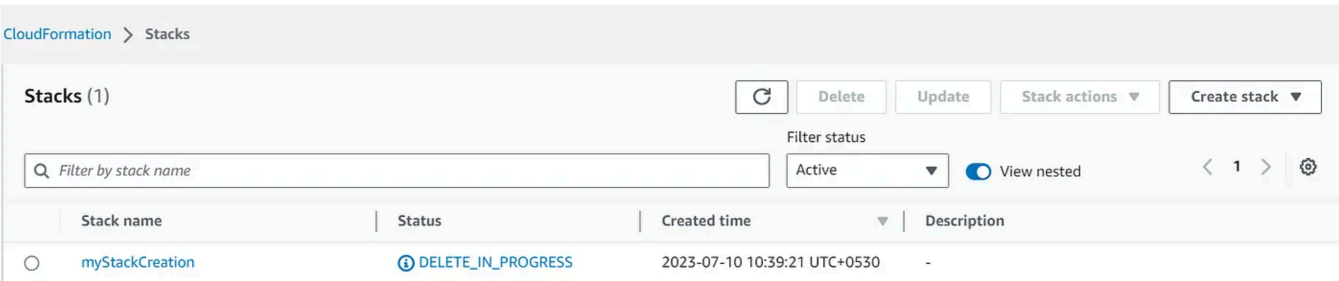


Update Completed

5. **Delete Stack:** Deletes the existing Stack.

```
aws cloudformation delete-stack --stack-name myStackCreation
```

```
C:\Users\samounika\Desktop>aws cloudformation delete-stack --stack-name myStackCreation
```



Deletion Image

That's it! We have created, updated and deleted the stack through AWS CLI.

If you find this article helpful, Please do follow and show your support 😄

Thank-you for reading!😄

— Satya Mounika Mushini